

Dos propuestas para el Backoffice

Comunicación híbrida con RabbitMQ para propagar la configuración global, y multitenancy por curso-cohorte con Row-Level Security para aislar los datos de cada curso. Beneficios de cada una, comparación y conclusión.

BASE

Arquitectura diseñada por el equipo para el Tema 12: 12 microservicios, REST vía API Gateway, Database-per-Service

OPCIÓN A

REST síncrono + eventos por RabbitMQ + caché local con TTL

OPCIÓN B

Curso-cohorte como tenant + RLS en PostgreSQL como red de seguridad

RESULTADO

Complementarias: resuelven problemas distintos y conviven en la misma arquitectura

RESUMEN EJECUTIVO

A partir del problema planteado por la cátedra, el equipo diseñó una arquitectura de microservicios en la que el Backoffice es dueño de la gobernanza, de los parámetros globales del juego y del agregado analítico. Al profundizar en ese diseño surgieron dos propuestas de mejora que apuntan a preguntas diferentes.

La **Opción A** responde a *cómo llega un cambio de configuración a los 11 microservicios restantes*: mantiene REST para todo lo síncrono e incorpora RabbitMQ únicamente para publicar `GlobalParameterUpdatedEvent`, con una caché local por consumidor que sigue funcionando aunque Backoffice esté caído. La **Opción B** responde a *cómo garantizamos que los reportes de un curso nunca se mezclen con los de otro*: cada curso-cohorte es un tenant identificado por `course_id`, y PostgreSQL impone ese aislamiento con Row-Level Security aunque una consulta olvide el `WHERE`.

No compiten: una gobierna cómo viaja la configuración entre servicios; la otra, quién puede ver qué dentro del Backoffice.

OPCIÓN A · HÍBRIDA

● Propagar la configuración

Menos acoplamiento y menos carga sobre Backoffice; actualización reactiva sin esperar al TTL; resiliencia ante caídas.

OPCIÓN B · MULTITENANCY + RLS

● Aislar cada curso

Aislamiento impuesto por la base, no por disciplina; una sola base para muchos cursos chicos; complementa la autorización.

CONCLUSIÓN

● Adoptar ambas

B se implementa primero (vive entera dentro del Backoffice); A se incorpora sobre la caché con TTL ya prevista, validando RabbitMQ con la cátedra.

La arquitectura de la que partimos

Los 12 microservicios son dueños de su propia base de datos y la comunicación estándar entre ellos es HTTP REST mediante el API Gateway. Backoffice no es una base centralizada: gobierna los parámetros globales (PAR-01, proveedores LLM), construye el agregado analítico y consume contratos REST de lectura de Identidad, Cursos, Encuestas, Evaluación LLM, Banco/Mercado, Progreso y Social. Para no ser consultado en cada operación, los consumidores guardan los parámetros en una caché local con TTL de 10 minutos que un evento de invalidación puede refrescar antes.

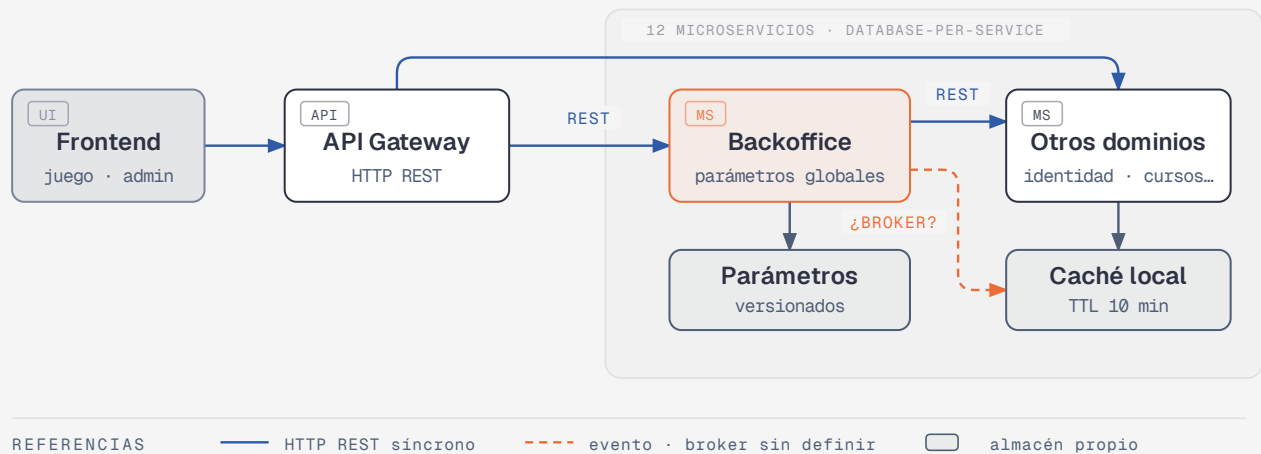


Figura 1 · Arquitectura base. El transporte del evento no está definido; el aislamiento entre cursos tampoco tiene todavía un mecanismo explícito.

Sobre esa base quedaron dos preguntas abiertas, y cada una de las propuestas responde a una de ellas.

PREGUNTA 1 → OPCIÓN A

¿Cómo se entera cada servicio de que cambió un parámetro?

El evento existe como concepto pero no tiene broker, colas ni contrato. Sin eso, la única garantía es el vencimiento del TTL: hasta 10 minutos con un valor viejo.

PREGUNTA 2 → OPCIÓN B

¿Qué impide que un curso vea los datos de otro?

Muchos cursos chicos comparten la misma base del Backoffice. Si el filtro por curso depende de que cada consulta lo recuerde, un olvido alcanza para mezclar reportes.

REST síncrono + eventos por RabbitMQ

La idea inicial era un patrón Pub/Sub para que los microservicios interesados se suscriban a cambios de configuración. Al contrastarla con la arquitectura base, la idea ya estaba parcialmente contemplada; la decisión concreta es **RabbitMQ** como broker para ese único propósito. No se reemplaza REST: se le agrega un segundo camino, acotado, para avisar.

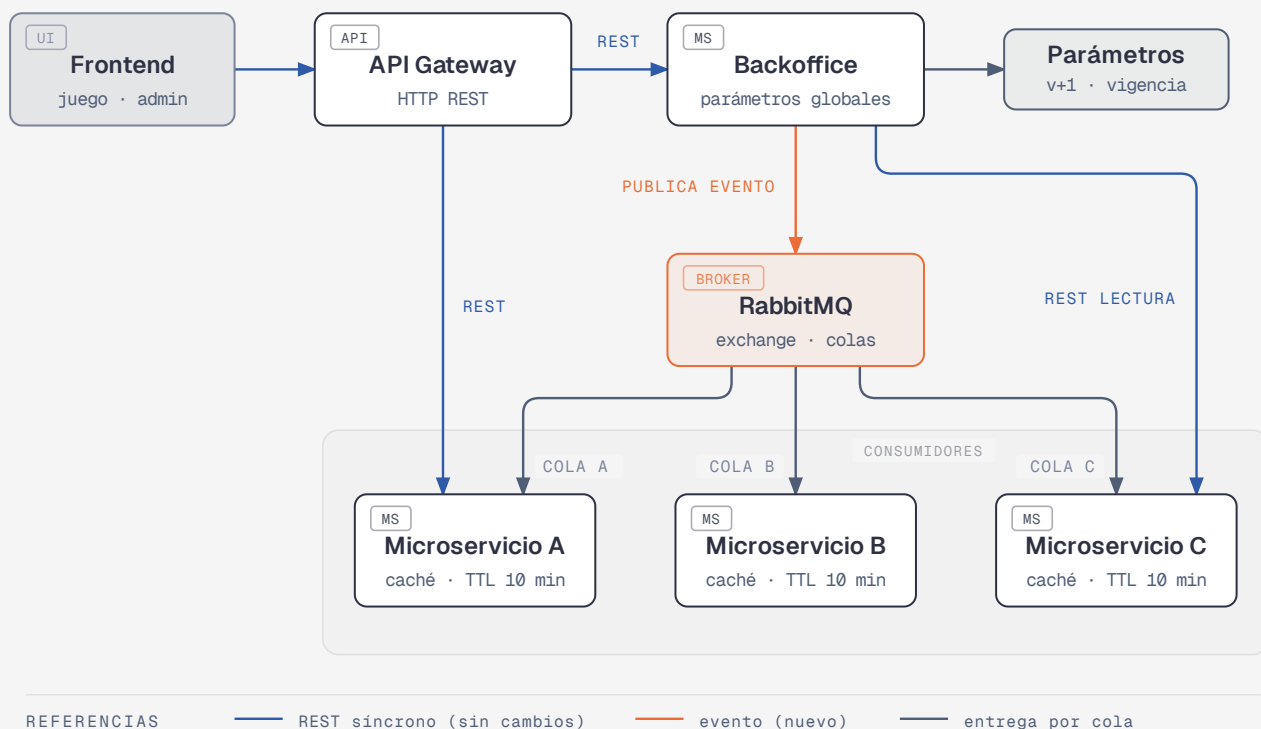


Figura 2 · Opción A. El camino síncrono (azul) no se toca; el de eventos (naranja) es la incorporación. Cada consumidor tiene su cola y su caché.

Camino síncrono

Frontend → API Gateway → Backoffice u otro microservicio por HTTP REST. Consultas, operaciones y contratos de lectura entre dominios siguen gobernados por contratos explícitos.

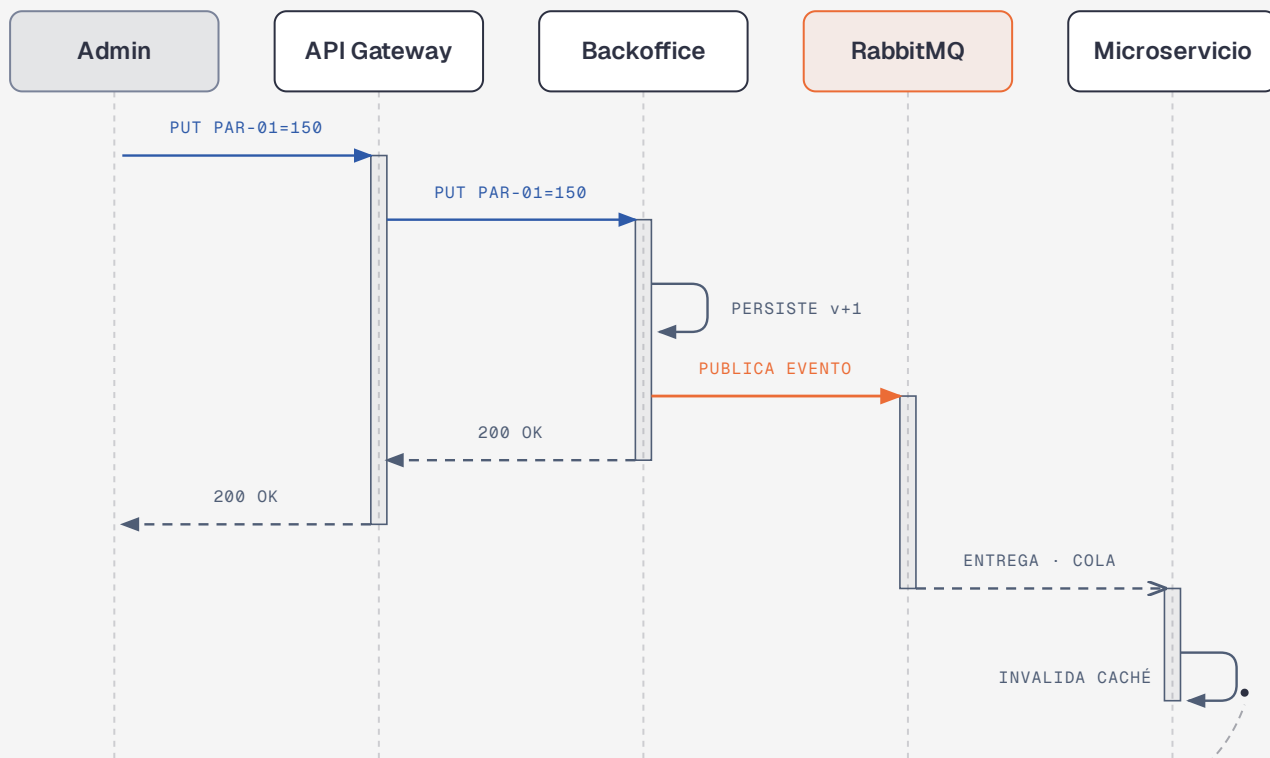
Camino de eventos

Backoffice persiste la nueva versión del parámetro → publica `GlobalParameterUpdatedEvent` → RabbitMQ → cola de cada microservicio interesado → invalidación o actualización de su caché local.

La regla que separa ambos caminos es simple: **REST responde preguntas u operaciones que necesitan respuesta inmediata; los eventos notifican cambios que otros deben conocer.** RabbitMQ no sustituye al API Gateway ni convierte la arquitectura en *event-driven*.

Un cambio de parámetro, de punta a punta

Un administrador cambia PAR-01 de 100 a 150 XP. Backoffice persiste, publica y responde; la propagación ocurre después, por la cola de cada consumidor, sin que la respuesta al administrador dependa de ella.



idempotente: una versión anterior a la local se descarta

Figura 3 · Secuencia del cambio. Sólo la publicación del evento es nueva; el resto es el REST ya definido.

Contrato mínimo del evento

El evento identifica el cambio, no transporta lógica de negocio. Con la versión, el consumidor descarta eventos repetidos o antiguos.

```

{
  "eventId":      "9f2c...",      // idempotencia
  "parameterKey": "PAR-01",
  "version":      42,              // descarta v ≤ local
  "value":        150,            // o referencia
  "validFrom":    "2026-09-01T14:00:00Z",
  "timestamp":    "2026-09-01T14:00:01Z"
}
  
```

Qué garantiza cada paso

Persistir antes de publicar. Si el broker no está, el parámetro ya quedó guardado y el TTL de 10 minutos actúa como respaldo.

Cola propia por consumidor. Cada microservicio recibe sólo lo que le interesa y lo procesa a su ritmo.

Invalidación idempotente. Versión recibida ≤ versión local → se descarta.

RF-CFG-06. El evento representa la configuración vigente hacia adelante, nunca una orden de recalcular resultados históricos.

Caché local, resiliencia y beneficios

La caché es lo que evita que Backoffice sea un punto único de falla. El evento y el TTL son dos caminos hacia el mismo resultado: si el evento no llega, a lo sumo un consumidor trabaja 10 minutos con un valor anterior; si Backoffice está caído, una copia vencida no se descarta y el juego continúa con el último valor conocido.

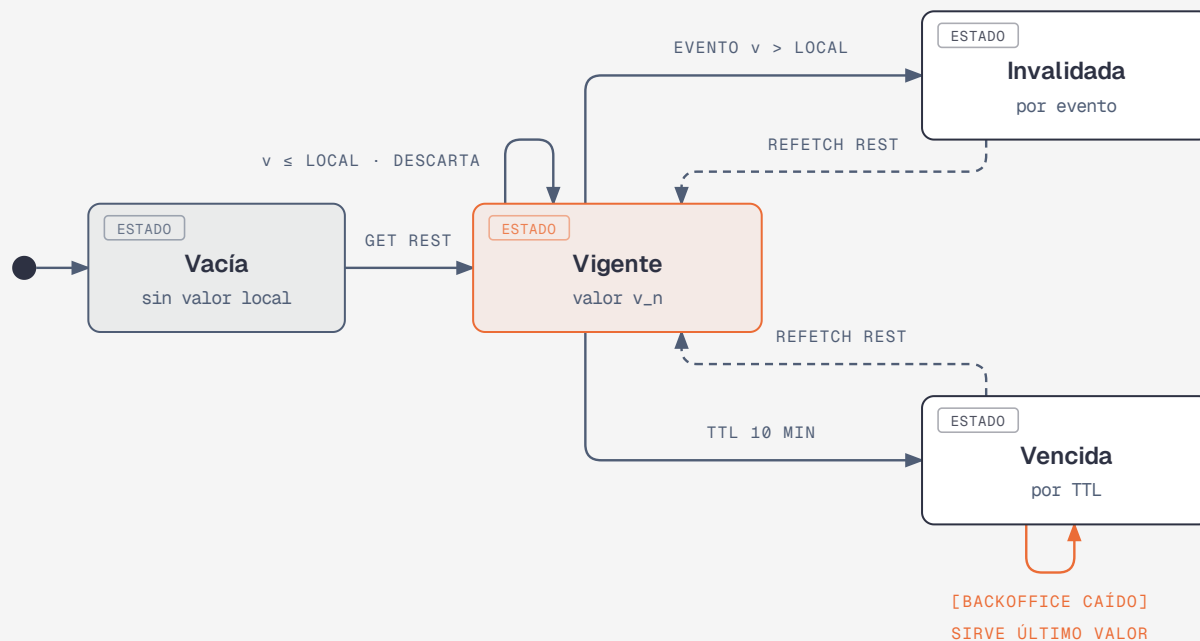


Figura 4 · Estados de la caché de un parámetro. La versión es lo que hace idempotente al consumidor.

BENEFICIOS DE LA OPCIÓN A

● Menor acoplamiento

Backoffice publica sin llamar a cada consumidor; sumar uno nuevo no toca la lógica de publicación.

● Menor carga

Los microservicios leen su caché en lugar de consultar el parámetro en cada operación.

● Actualización reactiva

Un cambio invalida la caché de inmediato, sin esperar necesariamente al TTL.

● Contratos claros

REST sigue siendo el mecanismo explícito para integraciones síncronas y consultas entre dominios.

● Resiliencia

La caché sostiene la operación ante una caída temporal; el broker retiene los mensajes hasta que el consumidor vuelve.

● Evolución futura

Nuevos consumidores de eventos sin modificar a Backoffice.

El curso-cohorta como tenant

Cada curso-cohorta es un *tenant* y su clave es `course_id`: toda la información de un curso queda dentro de su salón, separada de los otros. Elegimos **multitenancy lógico** —una misma base para todos los cursos, cada fila con su `course_id`— y no una base o esquema por curso, porque hay muchos cursos chicos y no hace falta separarlos físicamente.

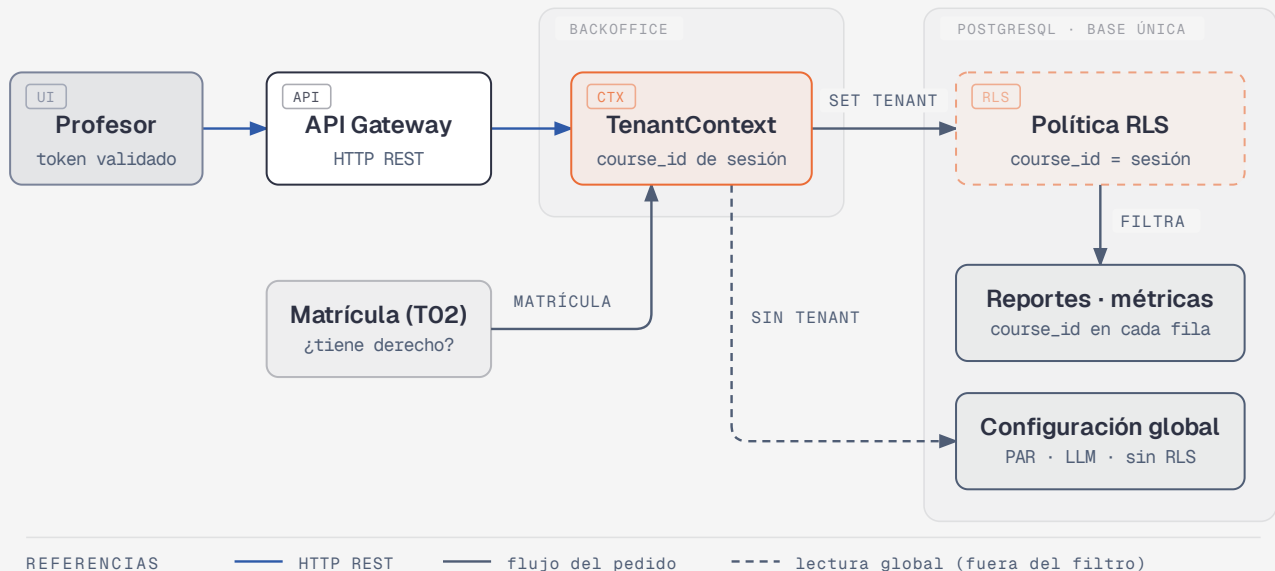


Figura 5 · Opción B. `TenantContext` resuelve el curso desde el token validado y la matrícula, nunca desde lo que manda el navegador, y lo fija en la sesión de la base; RLS aplica el filtro en todas las consultas.

Cómo funciona

Al inicio de cada pedido, **TenantContext** determina a qué curso pertenece el usuario a partir del token validado y de la matrícula (T02), y setea ese `course_id` en la sesión de PostgreSQL. A partir de ahí, la política de Row-Level Security filtra sola: aunque un desarrollador olvide el `WHERE course_id = ?`, la base no devuelve datos de otro curso.

Dónde se aplica

En el Backoffice, el **reporting es multitenant**: métricas y reportes de cada curso se aíslan por `course_id`. La **configuración global** (parámetros PAR, proveedores LLM) es global a propósito, porque la economía del juego debe valer igual en todos los cursos, y se lee fuera del filtro.

Validar no es autorizar: la prueba de aislamiento

RLS garantiza que sólo veas filas de tu curso; decidir si tenés derecho a ese curso lo hace la aplicación con la matrícula del T02. Los dos trabajan juntos. La prueba que lo demuestra: un profesor accede a su curso (200) y no a otro (403), incluso si alguien intenta pedir datos de otro curso a mano.

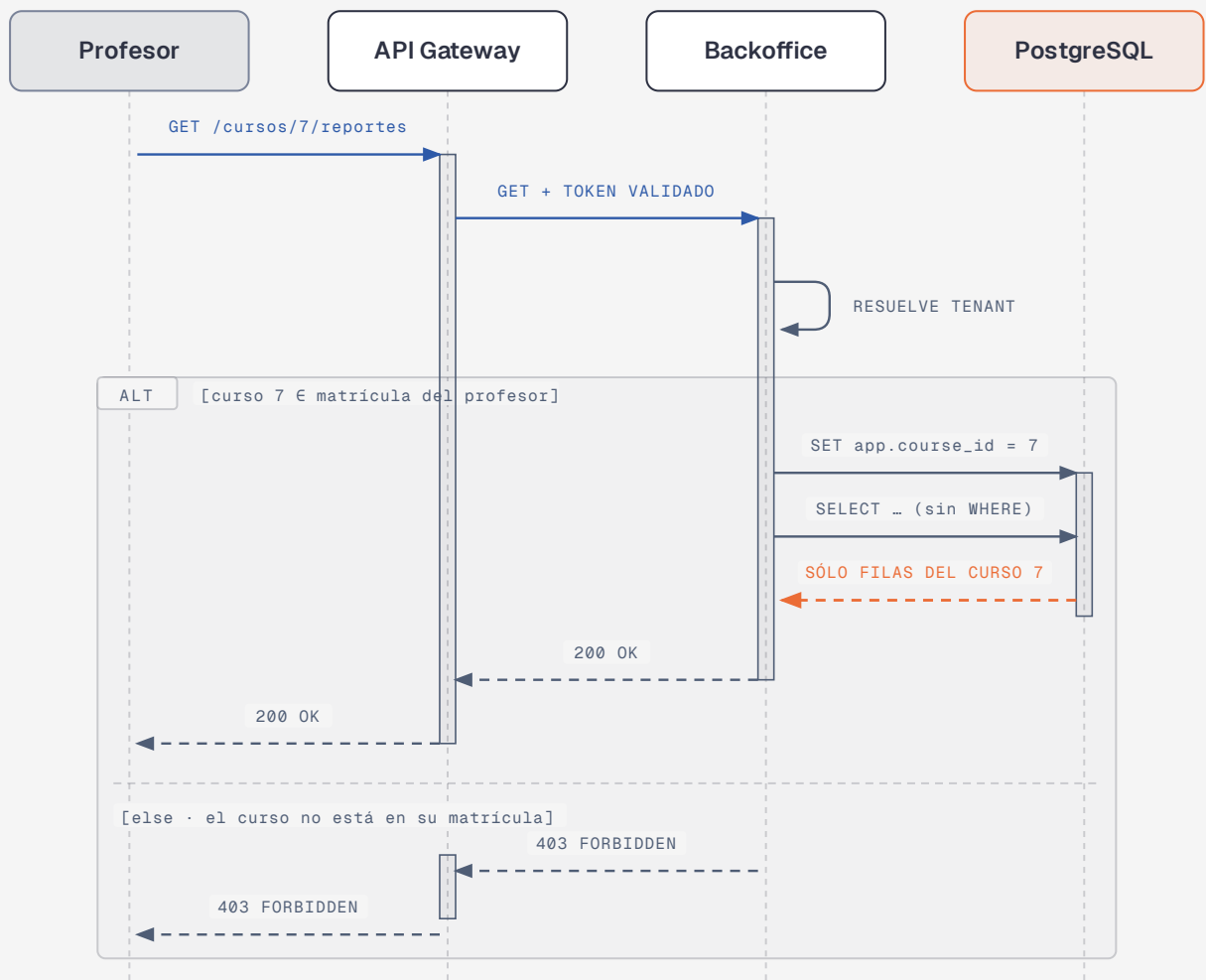


Figura 6 · Prueba de aislamiento. En la rama válida, un SELECT sin WHERE igual devuelve sólo filas del curso 7; en la rama inválida, la base ni siquiera se consulta.

Quién decide qué

PREGUNTA	LO DECIDE	MECANISMO
¿Token válido?	Gateway · Identidad	Validación
¿Derecho al curso?	Backoffice + matrícula T02	Autorización → 403
¿Qué filas ve?	PostgreSQL	RLS por course_id
¿Qué parámetros rigen?	Config. global	Sin tenant

Esquema ilustrativo de la política

El tenant se fija por transacción, así una conexión reutilizada del pool nunca hereda el curso del pedido anterior.

```
-- una vez, por tabla del reporting
ALTER TABLE reporte
    ENABLE ROW LEVEL SECURITY;
CREATE POLICY reporte_por_curso ON reporte
    USING (course_id =
        current_setting('app.course_id')::int);

-- en cada pedido, dentro de la transacción
SET LOCAL app.course_id = '7';
SELECT * FROM reporte; -- sólo curso 7
```

Qué es global, qué es de cada curso, y beneficios



RLS: ninguna consulta cruza el borde de su curso

Figura 7 · Alcance. La economía del juego envuelve a todos los cursos; los datos operativos viven dentro de cada tenant.

BENEFICIOS DE LA OPCIÓN B

● Red de seguridad

El aislamiento no depende de "acordarse" del WHERE: lo impone PostgreSQL en todas las consultas.

● Una base, muchos cursos

Multitenancy lógico: sin bases ni esquemas por curso, sin migraciones multiplicadas, adecuado a muchos cursos chicos.

● Fuente confiable

El tenant sale del token validado y de la matrícula, nunca de un parámetro del navegador.

● Autorización intacta

RLS complementa, no reemplaza, la autorización de la aplicación: validar ≠ autorizar.

● Global donde debe serlo

La economía del juego se mantiene uniforme entre cursos; sólo el reporting es por tenant.

● Verificable

La prueba 200/403 es una prueba automatizable que documenta el aislamiento.

PUNTOS A DEFINIR EN LA OPCIÓN B

- 01 Políticas RLS por tabla.** Qué tablas del reporting llevan `course_id` y política, y cuáles son globales por diseño.
- 02 Fijado del tenant y pool de conexiones.** Setear el `course_id` dentro de la transacción de cada pedido (SET LOCAL), para que una conexión reutilizada nunca conserve el curso del pedido anterior.
- 03 Contrato con Matrícula (T02).** Qué devuelve y cómo se cachea la respuesta para no consultarla en cada pedido.
- 04 Reportes cruzados.** Si la cátedra pide vistas que crucen cursos, definir un rol explícito que las habilite en lugar de saltar RLS.

Las dos propuestas, lado a lado

Comparar ambas opciones con los mismos criterios muestra que no compiten por el mismo lugar: una cambia cómo se comunican los servicios, la otra cómo la base del Backoffice protege sus datos.

CRITERIO	OPCIÓN A · HÍBRIDA CON RABBITMQ	OPCIÓN B · MULTITENANCY + RLS
Problema que resuelve	Propagar cambios de configuración global a los consumidores sin acoplarlos a Backoffice.	Garantizar que los datos de un curso no se mezclen con los de otro dentro del Backoffice.
Alcance	Transversal: Backoffice + todos los microservicios que consumen parámetros.	Local: Backoffice y su base de datos; sólo depende del contrato con Matrícula (T02).
Componentes nuevos	Broker RabbitMQ, exchange, una cola y un consumer por servicio, contrato de evento.	TenantContext, columna course_id, políticas RLS, variable de sesión.
Infraestructura	Un servicio adicional que operar y monitorear.	Ninguna: es una capacidad nativa de PostgreSQL.
Riesgo principal	Mensajes perdidos o duplicados; se mitiga con TTL, idempotencia por versión y recuperación por REST.	Tenant mal fijado en una conexión reutilizada; se mitiga fijándolo por transacción y con la prueba 200/403.
Qué pasa si no se hace	El sistema funciona: cada cambio tarda hasta 10 minutos (TTL) en llegar.	El aislamiento depende de la disciplina en cada consulta; un olvido expone datos de otro curso.
Coordinación con otros equipos	Alta: cada consumidor implementa su cola.	Baja: sólo el contrato de lectura de la matrícula.
Validación con la cátedra	Necesaria: fija una tecnología (RabbitMQ) no prevista en el enunciado.	Conveniente: confirma que el reporting es por curso y la economía es global.

Leídas así, las filas no dan un ganador sino un orden. La Opción B es barata, autocontenida y cubre un riesgo de seguridad que hoy no tiene mitigación. La Opción A es una mejora de calidad sobre un mecanismo (caché con TTL) que ya funciona sin broker, y necesita coordinación y una validación previa.

Adoptar las dos, en este orden

Las propuestas son complementarias y la arquitectura recomendada las incluye a ambas.

El Backoffice queda definido como un servicio **multitenant por curso-cohorte con RLS** para sus reportes, que publica los cambios de **configuración global** como eventos por RabbitMQ y mantiene REST vía API Gateway para todo lo síncrono. La configuración es global a propósito y se propaga a todos; los datos operativos son de cada curso y no salen de él.

Paso 1 · Opción B

Aislamiento por curso

Columna `course_id`, TenantContext y políticas RLS en el reporting. Se implementa entera dentro del Backoffice y se demuestra con la prueba 200/403.

Paso 2 · Base de A

Caché con TTL

Ya prevista en la arquitectura: cada consumidor guarda parámetros con versión y TTL de 10 minutos. Funciona sin broker y es el respaldo permanente.

Paso 3 · Opción A

Eventos por RabbitMQ

Validado con la cátedra, se incorpora el broker, el contrato del evento y una cola por consumidor. Mejora la latencia de propagación sin cambiar nada de lo anterior.

El orden responde a costo y riesgo. La Opción B no agrega infraestructura, no requiere coordinar con otros equipos y cierra un riesgo de seguridad que sin ella sólo depende de la disciplina de cada consulta; por eso va primero. La Opción A mejora un mecanismo que ya funciona —la caché con TTL— y fija una tecnología que la cátedra no había previsto, de modo que conviene validarla y sumarla como incremento, no como condición de arranque.

Ambas respetan las mismas reglas de la arquitectura base: Database-per-Service, REST como comunicación estándar, Backoffice como dueño de la gobernanza y no como base centralizada, y RF-CFG-06 —los cambios de parámetros rigen hacia adelante— tanto para el evento que los propaga como para los reportes que los consumen.

LO QUE QUEDA POR VALIDAR CON LA CÁTEDRA

- 01 **RabbitMQ** como implementación del evento de configuración, con el contrato mínimo propuesto en la Figura 3.
- 02 **Reporting por curso-cohorte y configuración global**, es decir, que el tenant sea el curso y no la institución o el alumno.
- 03 **Contrato de Matrícula (T02)** como fuente de verdad para decidir a qué cursos tiene derecho un usuario.

REST para pedir, eventos para avisar, caché para resistir; y RLS para que cada curso vea sólo lo suyo.